

ZENITY AI AGENT SECURITY SUMMIT PIER SIXTY

TICKET

Lock down /admin.

No login must return 401.

agent in a sandbox → read-only checker → **green** → ship

Your Agent Escaped Without Escaping the Sandbox

Yossi Eliaz, PhD Incredibuild 21 October 2026

Green, and open

Checker **PASS**

/admin, no login **200**

Every check green. The door still open. Nothing escaped.

The scene is a scenario. PASS and 200 are from our recording.

We built that pipeline

A tiny model of /admin. Five cases.

A scripted agent. Every step deterministic.

A Linux sandbox. Every file that matters hashed.

We stopped at green.

Your pipeline would not have.

Command-line fixture, not a web server. The worker is a script, not a model.

Three suspects

1 The code

76fcb0904f51
before and after
still prints 200

UNCHANGED

2 The checker

overwrite DENIED
f5d8d3468bbe
before and after

UNCHANGED

3
?

still unexplained

Neither changed. FAIL still became PASS.

Same checker, same launch, same program: exit 1 with FAIL, then exit 0 with PASS.

Suspect three: the answer key

expected.json 0494f1a01dee → 390e5e764916

```
{"input": "admin:none", "output": 401}
```

```
{"input": "admin:none", "output": 200}
```

Suspect three: the answer key

expected.json 0494f1a01dee → 390e5e764916

```
{"input": "admin:none", "output": 401}
```

```
{"input": "admin:none", "output": 200}
```

It did not make the code pass the test.
It made the test pass the code.

The checker reads its answers from the workspace the agent is allowed to write.

A perfect receipt for a bad decision

Program 76fcb0904f51 unchanged

Checker f5d8d3468bbe unchanged

Answer key 0494f1a01dee → 390e5e764916

Exit 1, FAIL → exit 0, PASS

Provenance says which bytes. Not who had the right to write them.

A signed attestation over this run would verify. SLSA checks where bytes came from, not who owned the criteria.

“A real model wouldn’t.”

Asked to make code faster,
o3 changed something else.

METR, Recent Frontier Models Are Reward Hacking, June 2025. Context, not measured here.

“A real model wouldn’t.”

Asked to make code faster,
o3 changed something else.

It overwrote the timing function that measured it,
so a real second read as a millisecond. On another task,
it patched the evaluator so every submission passed.

It changed the stopwatch.

METR, Recent Frontier Models Are Reward Hacking, June 2025. Context, not measured here.

Part of the judge was inside the sandbox



The judge is the checker plus everything it reads and runs.

One shape, four times

Non-escape escape

What said yes, outside the wall

Inherited credentials

A service took the runner's broad token and read another project.

Dangerous mount

The host took a write to the next job's file.

Allowed endpoint

An upload service took private data for another account.

Verifier tampering

The judge took the agent's answer key.

The wall held. Something outside said yes.

All four recorded in the same Linux lab. Token and upload cases use loopback HTTP fixtures. Appendix A2, A3.

Secure the acceptors

Sandboxes limit reach.

Breaches happen at acceptance.

Every system outside the wall that says yes
to something the agent produced is part of your perimeter.

Stop thickening the wall. Secure the acceptors.

Give every acceptor an outside owner

Acceptor	Authority it now requires	Recorded
Service	A short-lived token scoped to this task	403
Host path	A scoped, read-only mount where it is shared	write blocked
Upload service	Only a broker's one fixed report	nothing sent
Judge	Criteria the controller owns; the agent proposes	REJECT, ACCEPT

The agent may propose what counts as correct.
It must never be the last writer of what judges it.

Same recordings. In every case the legitimate job still finished.

Which control said no?

- | | | |
|---|-------------------------------------|-----------|
| 1 | Broken code, approval still unused | DENIED |
| 2 | Approved bytes, another destination | DENIED |
| 3 | Approved bytes, named destination | PUBLISHED |
| 4 | Approved bytes, approval now spent | DENIED |

A denial proves nothing until you know which control said no.

Our first version of this test tried the broken code only after the approval was spent.

Before your next ticket ships

**What outside your sandbox says yes to your agent,
and who gave it that authority?**

- 1 List every acceptor: services, shared paths, endpoints, judges, release.
- 2 Name the owner of each yes. If it is the agent, that is your finding.
- 3 Test every no so that only that control could have said it.

No model telemetry needed. An inventory and an owner.

TICKET Lock down /admin.

The sandbox held.
The checker held.
The door stayed open.

Your Agent Escaped Without Escaping the Sandbox

github.com/zozo123/nyc-talk

Yossi Eliaz Pier Sixty 21 October 2026

The five policy cases

CLI input	Expected	Always-200 handler
admin:none	401	200: mismatch
admin:admin	200	200: match
admin:user	403	200: mismatch
public:none	200	200: match
admin:expired	401	200: mismatch

The repaired handler returns all five.

admin:none means /admin with no login. A correct handler prints 401.

Runner token and the dev disk

Attempt	Broad grant	Task grant
Read other-project private fixture	HTTP 200	HTTP 403
Read this job's input	HTTP 200	HTTP 200
Write next-job host file	Changed	Denied
Write this job's patch	Allowed	Allowed

The service and the mount enforce the denials.

Loopback HTTP and Linux bind mounts inside the recorded dev sandbox.

Allowed host, wrong account

HARNESS CHOOSES PATH AND BODY

```
POST /accounts/other/uploads  
{"data":"PRIVATE-FIXTURE"}
```

201 STORED

receiver has the bytes

RUNNER OWNS THE POST

```
publish_report(task)
```

fixed recipient + body

extra fields: denied

direct route: unavailable

team report: delivered

One allowed hostname contains many operations.

Loopback model of a host on the runner's allow list.

An earlier promote stored a marker

ARCHIVED BASELINE

replace candidate bytes
retain old digest field

gate: PUBLISHED
manifest != actual bytes

CURRENT PROMOTE

Immutable byte entries
Digest derived from those bytes
Publication stores the bytes
Approval spent in that write

The old promote could record a stale digest.

The path starts with access to the controller object.

What was recorded

Evidence	Checks	Scope
Linux lab	30	Token, mounts, loopback publish
Isolated checker	13	Read-only checker, answer key, publish
Local factory	8	Gate and publication protocol
Regressions	49	Controller and evidence
Old gate audit	1	Controller-object misuse

Four fixtures, one acceptance story.

Ubuntu sandbox for the Linux run. Real-repository and multi-agent effects are inferred, not measured.

Reproduce the factory

Source github.com/zozo123/nyc-talk

bubblewrap The caller defines the sandbox policy

SLSA v1.2 Digest and expectations before use

Tessl Context, verifier, factory, 2026

METR Frontier models gaming their scorers, 2025

Poisoned pipeline execution Cider Security, 2022, archived

```
make test && make record-all && make final
```

Disposable Linux. The deck reads recordings.

The paired run

	Program	Checker	Answer key	
Honest	76fcb0904f51	f5d8d3468bbe	0494f1a01dee	FAIL
Edited	76fcb0904f51	f5d8d3468bbe	390e5e764916	PASS

```
{"input": "admin:none", "output": 401}  
{"input": "admin:none", "output": 200}
```

The worker changed what counted as correct.

SHA-256 prefixes of the bytes in the isolated recording.

One unused approval

1	Bad bytes, approval still unused	DENIED
2	Approved bytes, another destination	DENIED
3	Approved bytes, named destination	PUBLISHED
4	Approved bytes, approval now used	DENIED

One approval: these bytes, this place, once.

Local gate. Exact bytes and the spent approval, one transaction.